

I) CONTEXTE

Ce **nouveau projet** servira de support à l'examen de seconde session. Le respect des consignes est donc déterminant pour la réussite de cette Unité d'Enseignement ([INDJ0003-A-a] et [INDJ0003-B-a] pour 2 crédits ECTS). L'étude préliminaire est remplacée par une grille d'auto-évaluation qu'il vous faudra compléter et joindre au dépôt du projet.

II) CONSIGNES POUR LA REMISE

Compressez l'ensemble de votre arborescence dans une archive au format :

<VotreNomEnMajuscules>_<VotrePrénom>.zip

Ex : BRACAME_Edouard.zip

La taille de votre archive devra être **<= 200 Mo**.

Échéance

Vous devez remettre votre dépôt avant le **lundi 10 août 2026 à 18h00**,
délai de rigueur, l'heure du serveur Moodle faisant foi.

III) AVERTISSEMENTS

1. **Le travail à réaliser est une production personnelle.** Si cette consigne n'est pas respectée (*production partiellement ou entièrement copiée, réalisée à plusieurs étudiants, par IA ...*), **la note sera réduite pour correspondre au mieux à la production de l'étudiant.**
2. En cas de défaillance de Moodle, **et uniquement dans ce cas**, votre fichier est à envoyer par mail avec votre adresse étudiante à : benoit.burlion@hers.be , **avant le délai imparti** via le service : « Swiss Transfert », <https://www.swisstransfer.com/fr-be>
3. **Seuls les travaux remis dans le délai imparti seront acceptés.**
4. **Si votre travail n'est pas remis ou hors délai, vous ne pourrez pas vous présenter à l'examen.**
5. **Seuls les étudiants inscrits administrativement à la seconde session pourront accéder à l'examen.**
6. **L'ensemble des exigences de cet énoncé seront évaluées dans la note finale en complément des questions d'examens.**

IV) OBJECTIF DE L'EXAMEN

Dans le cadre de ce projet, l'objectif principal de l'examen est simple : prouver que vos bases de code (*productions et tests unitaires*) sont propres et « malléables ». Plusieurs modifications vous seront demandées, et l'une d'entre elles (*la question principale*) appellera un changement significatif.

V) RECHERCHES DOCUMENTAIRES ET USAGE DE « L' I.A. »

Nous souhaitons que ce projet vous permette d'explorer de nouvelles notions. Il vous faudra donc effectuer quelques recherches (*documentations officielles, ouvrages techniques, forum d'aide ...*)



Je n'ai en aucune
manière utilisé d'IA

Toutefois, nous soulignons ici et sans aucune ambiguïté, le distinguo entre une recherche d'information et l'usage de « l'I.A. » pour **générer du contenu**.

Par « *contenu* », nous entendons tout ce qu'une « I.A. » est capable de générer. **Dans le cadre de ce projet, l'usage de « l'I.A. » est interdit !**

Vous devrez nous remettre une attestation sur l'honneur lors du dépôt final.

Reprendre le code source d'autres développeurs (*notion de « maintenance logicielle » sur du code « hérité / Legacy »*) est probablement l'une des tâches les plus difficiles de notre métier. Nous vous conseillons vivement de rédiger vous-même l'intégralité des lignes de votre code source (*abstenez-vous de faire du copier-coller, même depuis la documentation officielle, ne laissez aucune zone d'incompréhension*), sous peine de vous retrouver rapidement en fâcheuse situation.

Nouvelle technologie, nouveaux enjeux

Ne confondez pas analyste-développeur et « prompt engineer ». Le premier est un métier qui requiert de **solides compétences**, tandis que le second relève davantage du « buzzword ». A notre connaissance, aucun employeur n'est à ce jour prêt à rémunérer un collaborateur pour copier-coller des réponses générées par une IA. En réalité, l'équation économique **joue en défaveur des profils juniors**. Là où les recruteurs demandaient hier : « Pourquoi vous plutôt qu'un autre ? », il vous faudra peut-être répondre demain à la question : « Pourquoi vous plutôt qu'une IA ? ».

Nous vous invitons donc à vous informer afin de mieux comprendre **pourquoi l'usage de l'IA est à proscrire dans le cadre de vos études**, afin justement d'accroître vos compétences, et d'appréhender comment utiliser ce nouvel outil de manière pertinente par la suite (*voir vidéo ci-contre sur votre temps libre*).



VI) Projet

Problématiques

L'importance de la disponibilité des services informatiques ne cesse de s'accroître dans notre société contemporaine. Par exemple, il suffit que notre plateforme *Moodle* soit à l'arrêt pour que nos activités pédagogiques soient fortement impactées. Pour limiter au strict minimum le temps de dérangement, l'équipe de maintenance doit être alertée au plus vite en cas de défaillance. Plusieurs solutions existent et proposent généralement deux volets : un affichage de l'état des machines et services surveillés ainsi qu'un système de notification en cas de défaillance.

Limites et contexte pédagogique

Afin d'éviter tout incident avec des tiers, nous limiterons strictement ce projet **au réseau local** en IPv4. Pour ne pas complexifier le programme à réaliser, nous testerons uniquement la réponse au *Ping* tout en étant bien conscients que dans la plupart des cas cette vérification est soit :

- insuffisante : une machine peut très bien répondre au Ping et avoir son service indisponible,
- inapplicable : des mesures de sécurité peuvent être prises pour ne pas répondre à ces requêtes.

Néanmoins ce projet est suffisant pour mettre en pratique les notions abordées en cours.

Rappel technique

Ce projet peut tout à fait fonctionner localement sur votre machine. Il est par exemple possible d'interroger l'interface de retour (*loopback*) de votre carte réseau. Toujours en réseau local, vous pourriez interroger une machine virtuelle, votre passerelle, votre dns ...

Contraintes

La solution logicielle

1. fonctionnera sous Windows 10/11 (x64), sans élévation de privilège,
2. ne nécessitera pas d'installation (*mode portable*¹),
3. sera exclusivement codé en C# sous Visual Studio 2026 (*dans sa dernière version à jour*),
4. n'utilise aucune dépendance externe, uniquement les classes natives .Net et Ms test,
5. sera codé dans le paradigme orienté objet.

Le code source et ses spécifications

6. seront rédigés en anglais, ne comporteront pas de défaut, et éviteront tout gaspillage mémoire ou temporel,
7. devront être facilement modifiables,
8. respecteront les bonnes pratiques en termes de lisibilité, ainsi que les conventions suivantes :
 - 8.1. nommage en C# <https://tinyurl.com/mun8a4ay>,
 - 8.2. fichiers sources inférieurs à 200 lignes,
 - 8.3. méthodes inférieures à 80 lignes (*spécifications et commentaires inclus*),

Agencement du projet

9. La solution sera composée de trois projets :
 - 9.1. une application de configuration,
 - 9.2. une application principale,
 - 9.3. les tests unitaires.
10. **Seules les classes présentes dans vos diagrammes UML seront implémentées.** Il s'agit donc de réaliser les diagrammes de conception et d'analyse pour les deux applications de production. Vous pouvez créer ces diagrammes UML avec l'outil de votre choix, mais ils doivent être regroupés dans le fichier analyse.pdf.
11. Trois dépôts locaux Git seront présents (*un pour chaque projet*). Ils contiendront la gestion des différentes versions de vos bases de code et respecteront la convention Karma :
<https://tinyurl.com/sojcpoq>
Les dépôts seront alimentés par plusieurs « *commits* », **a minima 1 commit par classe codée.**
12. Le projet de tests unitaires contient au moins deux tests unitaires sur les méthodes clés de votre logiciel. Chacun de ces tests contient plusieurs assertions.

¹ Tout exécutable compilé doit pouvoir fonctionner sur une machine où le Framework .Net est installé sans pour autant qu'un IDE (*Visual Studio*) soit présent.

Exigences fonctionnelles

L'application de configuration permettra

13. de créer *(ou recréer si besoin)*, compléter, modifier ou remettre à zéro la configuration *(si elle existe)*.
14. de renseigner jusqu'à huit machines à surveiller en spécifiant pour chacune :
 - 14.1. son nom,
 - 14.2. son adresse IPv4,
 - 14.3. l'intervalle d'interrogation en secondes entre deux requêtes Ping,
sachant que **chaque machine peut avoir un délai d'interrogation différent**,
15. de renseigner les paramètres des notifications SMTP *(port et activation SSL inclus)*,
16. de remettre à zéro la configuration *(s'il existe une configuration, même partielle)*,
17. de sauvegarder automatiquement la configuration **dès qu'un champ est modifié**
(l'utilisateur ne devra donc jamais cliquer sur un quelconque bouton « save »),
18. d'afficher en permanence *(en haut à droite, et dès lors qu'une configuration existe)* la date de la dernière modification au format :
« Last change saved on : hh:mm jj/mm/aaaa »,
19. de quitter à tout moment *(même si la configuration est incomplète)*, sans action complémentaire,
20. de quitter et de lancer le programme principal *(si une configuration complète existe)*.

L'unique fichier de configuration

21. sera **local à la solution**, il se nommera : config.json et sera placé dans le répertoire settings [\(voir figure 1\)](#).
22. sera référencé par les **trois projets** ainsi que les **exécutables générés** en utilisant **des chemins relatifs à votre solution**. Le déplacement de votre solution *(au complet depuis sa racine)* vers un autre emplacement ou une autre machine n'engendrera ainsi **aucun problème d'accès à la configuration**.

L'application principale

23. vérifiera à chaque démarrage qu'un fichier de configuration est bien présent et qu'il contient au moins une machine à surveiller. A défaut, elle proposera de se fermer et de lancer le programme de configuration.
24. permet de surveiller chaque machine enregistrée dans la configuration. Pour surveiller une machine, le programme émet des requêtes ping, selon l'intervalle qui a été configuré via le point 14.3.
Selon la réponse obtenue (*ou pas*), l'application met à jour l'état de la communication avec chaque machine surveillée (*cfr point 33*). L'état d'une communication peut être :
- 24.1. communication **non initiée** :
état de départ où aucune requête ping n'a encore été émise,
 - 24.2. communication **fonctionnelle** :
une requête ping a été émise à la machine à surveiller et elle y a répondu,
 - 24.3. communication **non fonctionnelle** :
une requête ping a été émise à la machine à surveiller et elle n'y a pas répondu.
25. notifie l'utilisateur à l'aide de la configuration SMTP du point 15 dès :
- 25.1. que la communication est **non fonctionnelle**.
 - 25.2. que la communication **redevient fonctionnelle**.
- Attention : il n'y a pas de notification au démarrage ni quand la communication passe de non initiée à fonctionnelle.
26. adapte automatiquement la taille de son interface selon le nombre de machines à surveiller.
Nous surveillerons au maximum de 8 machines.
27. propose en permanence à l'utilisateur une option : « always on top », qui lorsqu'elle est active empêche toute autre fenêtre de se placer au-dessus (*et vice-versa*).

Exigences non fonctionnelles

Les deux applications

28. seront chacune composées **d'un seul formulaire** en mode graphique (*Windows Forms App*),

l'utilisation de fenêtre pop-up est donc interdite !

L'objectif est d'utiliser l'héritage sur une classe de type conteneur comme vu en cours².

29. sont en permanence réactives : il ne doit y avoir aucun gèle (*freeze*),

30. se placeront au démarrage au-dessus de celles déjà ouvertes.

31. seront **ergonomiques** et simple d'emploi, vous veillerez :

31.1. à ne pas afficher **d'éléments inutiles**.

Voici quelques mauvais exemples :

- afficher l'option de remise à zéro de la configuration alors qu'elle n'existe pas,
- afficher la date et l'heure du dernier changement alors que la configuration n'existe pas,
- afficher un bouton « *leave and start monitoring* » alors que la configuration n'existe pas,
- afficher des visuels de surveillance de « machines inexistantes » ou « vides ».

31.2. à la lisibilité de vos interfaces.

Gardez à l'esprit qu'il faudra probablement modifier vos interfaces en examen ...

32. proposent **des interfaces en anglais exclusivement**.

L'application principale

33. Elle comprend :

- un bouton qui lance l'application de configuration,
- un bouton quitter pour sortir de l'application en terminant proprement tous les sous-programmes qu'elle a lancés,
- un visuel de l'état de chaque machine surveillée (*voir point suivant*).

34. Le visuel d'une machine surveillée :

- affiche en permanence son nom et son IPv4,
- est en texte noir sur fond gris au démarrage, lorsque la communication est **non initiée**,
- est en texte noir sur fond vert lorsque la communication est **fonctionnelle**,
- est en texte blanc sur fond rouge lorsque la communication est **non fonctionnelle**,
- change d'apparence, selon les 3 points précédents, dès qu'un changement d'état de la communication est détecté.

² Pour réaliser une succession d'écrans au sein d'un même formulaire, il est possible d'afficher/masquer des conteneurs qui peuvent être par exemple de type « *Panel* » ou autre. Veillez à bien modéliser au préalable ces objets et à prévoir un gestionnaire comme : « *PanelsManager* » ou « *LayersController* » ...

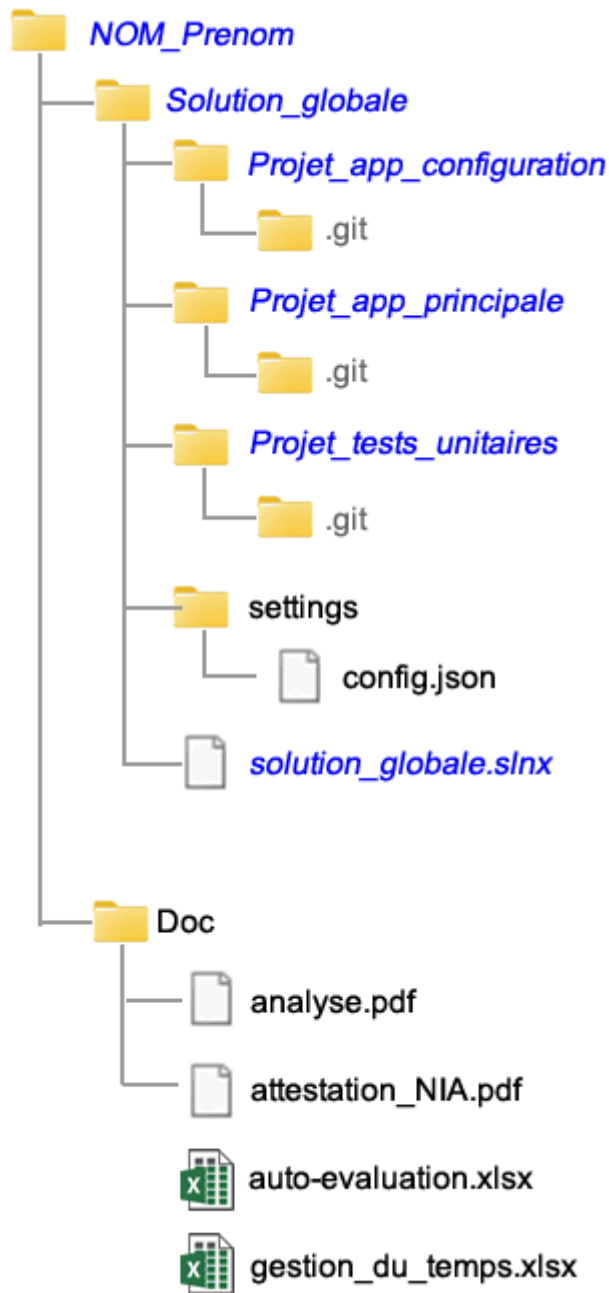
Gestion du temps

Afin d'affiner votre estimation du temps requis au développement d'une application, vous devrez compléter le fichier `gestion_du_temps.xls`. Il vous faudra donc estimer et mesurer le temps consacré à chacune des périodes de travail du projet, et ce dès le début de votre travail.

Lors de la phase de développement : **chaque commit dans votre dépôt local Git devra être en correspondance avec une tâche du fichier de gestion du temps**. Vous pouvez indiquer dans la colonne remarque l'identifiant du commit concerné.

Livrables

L'arborescence de votre dépôt **respectera l'agencement suivant :**



Les intitulés en **bleu italique** sont à remplacer par les vôtres.

=> Unique fichier de configuration

=> Le double clic sur votre fichier **.slnx** ouvrira Visual Studio et l'explorateur de solution contiendra bien les trois projets.

Figure 1 :

Modèle de l'arborescence du projet

Vous veillerez également à livrer :

- un exécutable par projet de production,
- des fichiers sources que **nous devons pouvoir compiler sans avertissements.**

vii) Quelques conseils

Si vous avez participé à la première session :

- faites votre bilan (*tableau à 4 colonnes : catégorie / positif / négatif / Remédiations*)
- ne reproduisez pas les mêmes erreurs.

Démarrez ce nouveau projet du bon pied :

- débutez le projet dès sa publication,
- commencez dès à présent à compléter le fichier de gestion du temps,
- imprimez cet énoncé et gardez-le sous la main,
- lisez cet énoncé plusieurs fois et notez au brouillon tout point obscur,
- commencez comme en première session par dessiner à main levée vos maquettes,
- faites un diagramme d'états pour vous aider à visualiser les différents états des communications ainsi que les transitions d'un état vers un autre. N'hésitez pas à le joindre à votre fichier analyse.pdf,
- dessinez vos diagrammes UML en réutilisant les notions abordées en cours (*principe de la succession d'écrans pour la configuration ...*),
- tester l'usage du « ping » avec un prototype.

Tout au long du projet :

- complétez soigneusement et simultanément votre gestion du temps et vos dépôts git,
- complétez régulièrement et progressivement le fichier d'auto-évaluation,
- répondez de prime abord aux contraintes de l'énoncé,
- appliquez l'ensemble des bonnes pratiques apprises dans votre Bachelier,
- tenter d'abord de trouver par vous-même les solutions,
- prenez du plaisir à concevoir ce projet.

Enfin, avant de remettre votre projet :

- faites tester l'ergonomie des programmes à vos proches,
- testez la « *portabilité* » de votre solution en l'exportant vers une autre machine pour vous assurer que les trois projets compilent bien sans avertissements et que vos tests unitaires et fonctionnels passent.